05_timetable_api_ingestion

September 24, 2026

1 Phase 4A: Automatisierte Fahrplandatensammlung via DB-Timetables-API

Dieses Hilfsnotebook implementiert die informationstechnische Pipeline zum selektiven Abruf fahrplanmäßiger Soll-Verbindungen. Ziel ist der Aufbau einer persistenten, temporären Datenbasis für das bayerische Schienennetzwerkmodell unter Berücksichtigung restriktiver API-Rahmenbedingungen.

Der Datenabruf erfolgt über die offizielle REST-Schnittstelle *Timetables v1* des DB-API-Marketplaces. Da Fahrpläne systembedingt nur stündlich und pro Betriebsstelle extrahiert werden können, steuert dieses Modul eine automatisierte Ingestion-Schleife über die verifizierten bayerischen EVA-Nummern. Zur Vermeidung redundanter Netzanfragen werden die empfangenen XML-Rohdaten strukturiert im lokalen Verzeichnis `data/xml_cache/` persistiert, um die anschließende topologische Kanten-Modellierung vorzubereiten.

1.1 Berücksichtigung von API-Schranken (Rate-Limiting) und defensiver Test-Architektur

Die Timetables-Schnittstelle des DB-Marketplaces unterliegt einem strikten Kontingent von maximal **60 Abfragen pro Minute** (HTTP-Status 429). Zur Gewährleistung der Pipelinesicherheit auf der Remote-Infrastruktur wird eine rein modulare Funktionsarchitektur implementiert.

Anstatt ungesteuerte Massenabfragen auszuführen, wird der Abruf in isolierten Python-Funktionen gekapselt. Vor der netzweiten Schleifenaktivierung wird über empirische Stichproben-Zellen die reale Netzwerklaufzeit gemessen. Dies ermöglicht eine präzise Kalibrierung des Verzögerungsintervalls (Sleep-Timer), um die maximale Bandbreite des API-Kontingents zeiteffizient auszunutzen.

[5]:
```
import os
import requests
import time
from pathlib import Path
import pandas as pd
from dotenv import load_dotenv

from pyspark.sql import SparkSession
from pyspark.sql.types import *
import pyspark.sql.functions as F

import shutil
```

```
import xml.etree.ElementTree as ET

from project_config import (
    PROJECT_ROOT,
    XML_CACHE_DIR,
    ENV_FILE_PATH,
    STADA_API_RESULTS,
    TIMETABLE_API_RESULTS, # Ergebnis Speicherort
    get_project_relative_path
)

# Laden der API-Schlüssel aus der definierten ENV_FILE_PATH
if ENV_FILE_PATH.exists():
    load_dotenv(dotenv_path=ENV_FILE_PATH)
    print("ERFOLG: .env-Datei geladen.")
else:
    print("WARNUNG: .env-Datei fehlt!")

client_id = os.getenv("DB_API_CLIENT_ID")
client_secret = os.getenv("DB_API_CLIENT_SECRET")
```

```
ERFOLG: .env-Datei geladen.
```

1.2 Konfiguration des Untersuchungszeitraums für die Massenabfrage

[7]:
```
ZIEL_DATUM = "260923"
FOKUS_STUNDEN = ["17", "18"]
```

1.3 Definition der Kernfunktion für API-Requests

[8]:
```
def fetch_single_timetable_plan(eva_no, datum, uhrzeit, c_id, c_secret):
    """
    Führt einen solitären API-Request an die DB-Timetables-Schnittstelle aus.
    Gibt die komplette Response (Status-Code und Text) zurück.
    """
    url = f"https://apis.deutschebahn.com/db-api-marketplace/apis/timetables/v1/
 ↪plan/{eva_no}/{datum}/{uhrzeit}"
    headers = {
        "DB-Client-Id": c_id,
        "DB-Api-Key": c_secret,
        "accept": "application/xml"
    }
    try:
        response = requests.get(url, headers=headers, timeout=10)
        return response.status_code, response.text
    except Exception as e:
```

```
        return 999, str(e)
```

1.4 Daten-Persistierung und Idempotenzschutz (XML-Sicherung)

- Standardisierte Dateibenennung auf Basis der Adressierungsparameter ({Datum}-{Stunde}-{EVA})
- Optionaler Überschreibschutz zur Vermeidung redundanter I/O-Prozesse und zur Zeitsicherung

[9]:
```
def save_timetable_xml(status_code, xml_inhalt, eva_no, datum, uhrzeit,␣
↪ziel_ordner, overwrite=False):
    """
    Validiert die Antwort und speichert den Inhalt im Format:␣
↪{YYMMDD}-{HH}-{evaNo}.xml
    Schützt standardmäßig vor ungewollten Überschreibungen zur Zeitsicherung.
    """
    if status_code != 200 or not xml_inhalt or "<timetable" not in xml_inhalt:
        return False, None

    dateiname = f"{datum}-{uhrzeit}-{eva_no}.xml"
    physischer_pfad = Path(ziel_ordner) / dateiname

    # Idempotenzschutz: Wenn Datei existiert und overwrite=False, Zeit sparen␣
↪und überspringen
    if physischer_pfad.exists() and not overwrite:
        return True, f"{dateiname} (Bereits vorhanden - übersprungen)"

    try:
        with open(physischer_pfad, "w", encoding="utf-8") as f:
            f.write(xml_inhalt)
        return True, dateiname
    except Exception as e:
        print(f"Schreibfehler bei Datei {dateiname}: {e}")
        return False, None
```

1.5 Extraktion der bayerischen Referenzknoten aus Phase 3

[6]:
```
# Bayrische Stationen
print(f"Lade offizielle DB-Referenzdaten von:␣
↪{get_project_relative_path(STADA_API_RESULTS)}")
df_referenz_stada = pd.read_parquet(STADA_API_RESULTS)

# Erstellung der zwingend benötigten Variable für die Massenschleife
df_referenz_bayern = df_referenz_stada[df_referenz_stada["api_federal_state"]␣
↪== "Bayern"]

# Extraktion aller eindeutigen, gültigen bayerischen EVA-Nummern
```

```
fokus_massen_evas = [str(int(eva)) for eva in df_referenz_bayern["api_eva_nr"].
 ↪dropna().unique()]
```

```
Lade offizielle DB-Referenzdaten von: outputs/04_results.parquet
```

[18]:
```
df_referenz_stada.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 5461 entries, 0 to 5460
Data columns (total 11 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   api_station_name   5461 non-null   str
 1   api_city           5461 non-null   str
 2   api_zipcode        5461 non-null   Int64
 3   api_federal_state  5461 non-null   str
 4   api_category       5461 non-null   Int64
 5   api_eva_nr         5461 non-null   Int64
 6   api_is_main_eva    5461 non-null   bool
 7   api_long           5461 non-null   float64
 8   api_lat            5461 non-null   float64
 9   api_product_line   5448 non-null   str
 10  api_segment        5448 non-null   str
dtypes: Int64(3), bool(1), float64(2), str(5)
memory usage: 831.1 KB
```

1.5.1 Datenkritische Stichprobenanalyse und Validierung spezifischer Netzelemente

[109]:
```
passau2 = df_referenz_stada[df_referenz_stada.api_station_name == "Neustift(b␣
 ↪Passau)"]
```

[110]:
```
passau2
```

[110]:
```
Empty DataFrame
Columns: [api_station_name, api_city, api_zipcode, api_federal_state,
api_category, api_eva_nr, api_is_main_eva, api_long, api_lat, api_product_line,
api_segment]
Index: []
```

1.6 Lokale Massen-Ingestion für das bayerische Teilnetz

- Implementierung des Sicherheits-Timers (0,9s) zur Einhaltung des API-Schwellenwerts.
- Die Parameter `ZIEL_DATUM` und `FOKUS_STUNDEN` sind schnittstellenbedingt eng an die aktuelle Echtzeit gekoppelt.

  Architektonischer Hinweis: Aufgrund infrastruktureller Instabilitäten der Remote-Umgebung (Verbindungsabbrüche, Systemwartungen) wurde bewusst auf ein vollautomatisiertes Pipeline-Scheduling verzichtet. Das System operiert als stabiler, inkrementeller Datensammler, dessen Ingestion-Historie jederzeit beliebig erweitert werden

kann.

[27]:
```
# ============================================================================
# Operative Schleifenausführung
# ============================================================================

total_geplante_requests = len(fokus_massen_evas) * len(FOKUS_STUNDEN)
print(f"\nAPI-Massenstart: Starte Ingestion für {len(fokus_massen_evas)}␣
 ↪bayerische Bahnhöfe.")
print(f"Gesamtzahl geplanter Requests: {total_geplante_requests} |␣
 ↪Sicherheits-Timer: 0.9s\n")

start_massen_ingestion = time.perf_counter()
erfolgreiche_massen_downloads = 0
fehler_massen_downloads = 0
verarbeitete_requests = 0

# Die verschachtelte Schleife (Bahnhof -> Stunden)
for eva in fokus_massen_evas:
    for stunde in FOKUS_STUNDEN:
        verarbeitete_requests += 1

        # 1. API-Datenabruf
        status_code, xml_inhalt = fetch_single_timetable_plan(
            eva, ZIEL_DATUM, stunde, client_id, client_secret
        )

        # 2. Physisches Schreiben auf die Festplatte
        erfolg, datei = save_timetable_xml(
            status_code, xml_inhalt, eva, ZIEL_DATUM, stunde, XML_CACHE_DIR,␣
 ↪overwrite=True
        )

        if status_code == 200 and erfolg:
            erfolgreiche_massen_downloads += 1
        else:
            fehler_massen_downloads += 1

        # 3. Sicherheits-Timer zur strikten Einhaltung des 60er-Rate-Limits
        time.sleep(0.9)

        # Fortschrittsanzeige alle 50 Requests
        if verarbeitete_requests % 50 == 0:
            prozent = (verarbeitete_requests / total_geplante_requests) * 100
            aktuell_dauer = time.perf_counter() - start_massen_ingestion
```

```
            print(f"Fortschritt: {verarbeitete_requests}/
 ↪{total_geplante_requests} verarbeitet ({prozent:.1f}%) | Laufzeit:␣
 ↪{aktuell_dauer:.1f}s")

gesamtlaufzeit_massen_lauf = time.perf_counter() - start_massen_ingestion

print("\n" + "=" * 70)
print("ERFOLG: Bundeslandweite XML-Fahrplandatensammlung abgeschlossen!")
print(f"Erfolgreich im Cache gesichert:  {erfolgreiche_massen_downloads}/
 ↪{total_geplante_requests}")
print(f"Fehlgeschlagene Abfragen:        {fehler_massen_downloads}")
print(f"Gesamte Verarbeitungs-Laufzeit: {gesamtlaufzeit_massen_lauf / 60:.2f}␣
 ↪Minuten")
print("=" * 70)
```

```
API-Massenstart: Starte Ingestion für 931 bayerische Bahnhöfe.
Gesamtzahl geplanter Requests: 1862 | Sicherheits-Timer: 0.9s

Fortschritt: 50/1862 verarbeitet (2.7%) | Laufzeit: 59.0s
Fortschritt: 100/1862 verarbeitet (5.4%) | Laufzeit: 118.2s
Fortschritt: 150/1862 verarbeitet (8.1%) | Laufzeit: 176.9s
Fortschritt: 200/1862 verarbeitet (10.7%) | Laufzeit: 235.9s
Fortschritt: 250/1862 verarbeitet (13.4%) | Laufzeit: 295.2s
Fortschritt: 300/1862 verarbeitet (16.1%) | Laufzeit: 354.3s
Fortschritt: 350/1862 verarbeitet (18.8%) | Laufzeit: 413.6s
Fortschritt: 400/1862 verarbeitet (21.5%) | Laufzeit: 473.2s
Fortschritt: 450/1862 verarbeitet (24.2%) | Laufzeit: 532.6s
Fortschritt: 500/1862 verarbeitet (26.9%) | Laufzeit: 591.7s
Fortschritt: 550/1862 verarbeitet (29.5%) | Laufzeit: 650.6s
Fortschritt: 600/1862 verarbeitet (32.2%) | Laufzeit: 709.3s
Fortschritt: 650/1862 verarbeitet (34.9%) | Laufzeit: 768.2s
Fortschritt: 700/1862 verarbeitet (37.6%) | Laufzeit: 827.2s
Fortschritt: 750/1862 verarbeitet (40.3%) | Laufzeit: 904.9s
Fortschritt: 800/1862 verarbeitet (43.0%) | Laufzeit: 979.0s
Fortschritt: 850/1862 verarbeitet (45.6%) | Laufzeit: 1044.8s
Fortschritt: 900/1862 verarbeitet (48.3%) | Laufzeit: 1119.1s
Fortschritt: 950/1862 verarbeitet (51.0%) | Laufzeit: 1178.0s
Fortschritt: 1000/1862 verarbeitet (53.7%) | Laufzeit: 1236.8s
Fortschritt: 1050/1862 verarbeitet (56.4%) | Laufzeit: 1295.8s
Fortschritt: 1100/1862 verarbeitet (59.1%) | Laufzeit: 1354.9s
Fortschritt: 1150/1862 verarbeitet (61.8%) | Laufzeit: 1413.9s
Fortschritt: 1200/1862 verarbeitet (64.4%) | Laufzeit: 1472.7s
Fortschritt: 1250/1862 verarbeitet (67.1%) | Laufzeit: 1532.1s
Fortschritt: 1300/1862 verarbeitet (69.8%) | Laufzeit: 1590.7s
Fortschritt: 1350/1862 verarbeitet (72.5%) | Laufzeit: 1649.5s
Fortschritt: 1400/1862 verarbeitet (75.2%) | Laufzeit: 1708.3s
```

```
Fortschritt: 1450/1862 verarbeitet (77.9%) | Laufzeit: 1767.2s
Fortschritt: 1500/1862 verarbeitet (80.6%) | Laufzeit: 1826.2s
Fortschritt: 1550/1862 verarbeitet (83.2%) | Laufzeit: 1884.9s
Fortschritt: 1600/1862 verarbeitet (85.9%) | Laufzeit: 1944.3s
Fortschritt: 1650/1862 verarbeitet (88.6%) | Laufzeit: 2003.5s
Fortschritt: 1700/1862 verarbeitet (91.3%) | Laufzeit: 2062.3s
Fortschritt: 1750/1862 verarbeitet (94.0%) | Laufzeit: 2121.3s
Fortschritt: 1800/1862 verarbeitet (96.7%) | Laufzeit: 2180.3s
Fortschritt: 1850/1862 verarbeitet (99.4%) | Laufzeit: 2239.2s

======================================================================
ERFOLG: Bundeslandweite XML-Fahrplandatensammlung abgeschlossen!
Erfolgreich im Cache gesichert:  1856/1862
Fehlgeschlagene Abfragen:        6
Gesamte Verarbeitungs-Laufzeit: 37.56 Minuten
======================================================================
```

1.6.1 Empirische Leistungsdaten des Ingestion-Laufs

- **Untersuchungsumfang:** 2 Fokusstunden über 931 registrierte bayerische Bahnhöfe
- **Performance:** Die durchschnittliche Verarbeitungszeit pro Stunden-Slot lag stabil zwischen 17 und 20 Minuten
- **Datenintegrität:** Minimale Fehlquote (6 unkritische Timeouts bei 1.862 Anfragen); die Taktkonstanz ist netzweitzu 99,68 % gewahrt

1.6.2 Systemischen Restriktionen im Gesamtnetz

Die Kombination aus dem strikten API-Limit (60 Requests/Min.) und der stückweisen Abfrage-Architektur der Schnittstellen `fchg` (Fahrplanänderungen) und `rchg` (Echtzeit-Veränderungen) begründet die in Phase 2 und Phase 3 detektierte Fokussierung des Datensatzes auf die Kernkategorien 1 bis 5. Eine flächendeckende Echtzeit-Pipeline über sämtliche 5.461 deutschen Stationen ist unter den aktuellen Kontingentsbeschränkungen des DB-Marketplaces informationstechnisch nicht skalierbar.

1.7 Daten-Transformation: Ganzheitliches XML-Parsing des lokalen Caches

[7]:
```python
print(f"Starte Einlesen des XML-Caches aus:␣
 ↪{get_project_relative_path(XML_CACHE_DIR)}")
xml_dateien = list(Path(XML_CACHE_DIR).glob("*.xml"))

zuglaeufe_datenpool = []
start_parsing = time.perf_counter()

for datei_pfad in xml_dateien:
    dateiname_teile = datei_pfad.stem.split("-")
    if len(dateiname_teile) < 3:
        continue
    # Die EVA-Nummer des aktuellen Abfragebahnhofs steht am Ende des Dateinamens
```

```
    aktuelle_bahnhof_eva = int(dateiname_teile[-1])

    try:
        tree = ET.parse(datei_pfad)
        root = tree.getroot()
        aktueller_bahnhof_name = root.get("station", "Unbekannt")

        for zug in root.findall("s"):
            raw_id = zug.get("id")
            if not raw_id:
                continue

            id_segmente = raw_id.split("-")
            # Dreiteilung der DB-Fahrt-ID unter Berücksichtigung negativer Vorzeichen
            if raw_id.startswith("-"):
                p1_string = "-" + id_segmente[1]
                p2_string = id_segmente[2]
                p3_string = id_segmente[3]
            else:
                p1_string = id_segmente[0]
                p2_string = id_segmente[1]
                p3_string = id_segmente[2]

            # Extraktion der Basisdaten aus dem Transport-Layer (<tl>)
            tl_elem = zug.find("tl")
            train_category = tl_elem.get("c") if tl_elem is not None else None
            train_number = tl_elem.get("n") if tl_elem is not None else None

            # Extraktion des Arrival-Layers (<ar>) für Ankunftszeit und Vorgängerkette
            ar_elem = zug.find("ar")
            if ar_elem is not None:
                arrival_time = ar_elem.get("pt")
                arrival_platform = ar_elem.get("pp")
                line_ar = ar_elem.get("l")
                previous_path = ar_elem.get("ppth")
                # Extrahiert das letzte Element der Kette
                direkter_vorgaenger = [s.strip() for s in previous_path.split("|")][-1] if previous_path else None
            else:
                arrival_time, arrival_platform, line_ar, direkter_vorgaenger = None, None, None, None

            # Extraktion des Departure-Layers (<dp>) für Abfahrtszeit und Nachfolgerkette
            dp_elem = zug.find("dp")
```

```
            if dp_elem is not None:
                departure_time = dp_elem.get("pt")
                departure_platform = dp_elem.get("pp")
                line_dp = dp_elem.get("l")
                next_path = dp_elem.get("ppth")
                # Extrahiert das erste Element der Kette
                direkter_nachfolger = [s.strip() for s in next_path.
 ↪split("|")][0] if next_path else None
            else:
                departure_time, departure_platform, line_dp,␣
 ↪direkter_nachfolger = None, None, None, None

            zuglaeufe_datenpool.append({
                "station_eva_nr": aktuelle_bahnhof_eva,
                "station_name": aktueller_bahnhof_name,
                "fahrt_id_part1": p1_string,
                "fahrt_id_part2": p2_string,
                "fahrt_id_part3": int(p3_string),
                "train_category": train_category,
                "line_ar": line_ar,
                "line_dp": line_dp,
                "train_no": train_number,
                "plan_arrival": arrival_time,
                "arrival_platform": arrival_platform,
                "unmittelbarer_vorgaenger": direkter_vorgaenger,
                "plan_departure": departure_time,
                "departure_platform": departure_platform,
                "unmittelbarer_nachfolger": direkter_nachfolger
            })
    except Exception as e:
        continue

# Erstellung des ungefilterten, reinen Pandas-DataFrames
df_timetable_raw = pd.DataFrame(zuglaeufe_datenpool)
print(f"Ende: {len(df_timetable_raw)} Rohdatensätze in {time.perf_counter() -␣
 ↪start_parsing:.2f}s geladen.")
```

```
Starte Einlesen des XML-Caches aus: data/xml_cache
Ende: 32436 Rohdatensätze in 36.86s geladen.
```

[8]:
```
df_timetable_raw.head(10)
```

[8]:

	station_eva_nr	station_name	fahrt_id_part1	fahrt_id_part2	\
0	8000009	Ansbach	7287141852033567087	2609231254	
1	8000009	Ansbach	958115320951048375	2609230935	
2	8000009	Ansbach	-113756971786229411	2609230906	
3	8000009	Ansbach	-291665655105877819	2609231141	

```
4         8000009            Ansbach  -2596384342536523823     2609230750
5         8000009            Ansbach  -4910924940869376561     2609231100
6         8000009            Ansbach  -6614178894626142138     2609231232
7         8000009            Ansbach   8048313481640644275     2609231146
8         8000009            Ansbach  -3342761811808365604     2609231121
9         8000010  Aschaffenburg Hbf   5541256277927285538     2609231256

   fahrt_id_part3 train_category line_ar line_dp train_no plan_arrival  \
0               1              S     NaN      S4    39441          NaN
1              19            ARV    RE80    RE80    57026   2609231201
2               9             IC     NaN     NaN     2067   2609231200
3              12            ARV    RE80    RE80    57215   2609231245
4              13             IC     NaN     NaN     2162   2609231206
5              16            ARV    RE90    RE90    88617   2609231248
6               1              S     NaN      S4    39439          NaN
7              14              S      S4     NaN    39432   2609231227
8              14              S      S4      S4    39430   2609231202
9               1             RB     NaN    RB88    23315          NaN

  arrival_platform   unmittelbarer_vorgaenger plan_departure  \
0              NaN                        NaN     2609231254
1                3                  Triesdorf     2609231211
2                5                 Crailsheim     2609231201
3                4            Oberdachstetten     2609231254
4                5               Nürnberg Hbf     2609231208
5                5  Leutershausen-Wiedersbach     2609231250
6              NaN                        NaN     2609231232
7                1         Sachsen(b Ansbach)            NaN
8                1         Sachsen(b Ansbach)     2609231215
9              NaN                        NaN     2609231256

  departure_platform   unmittelbarer_nachfolger
0                  1         Sachsen(b Ansbach)
1                  3            Oberdachstetten
2                  5               Nürnberg Hbf
3                  4                  Triesdorf
4                  5                 Crailsheim
5                  5              Wicklesgreuth
6                 25         Sachsen(b Ansbach)
7                NaN                        NaN
8                  1  Leutershausen-Wiedersbach
9                  4   Aschaffenburg Hochschule
```

[10]:
```
display(df_timetable_raw.describe().round(2))
df_timetable_raw.info()
```

```
       station_eva_nr  fahrt_id_part3
count        32436.00        32436.00
```

```
mean       8005010.45            9.73
std          13113.61            7.43
min        8000009.00            1.00
25%        8001621.00            4.00
50%        8004008.00            8.00
75%        8004507.00           14.00
max        8098263.00           35.00
```

```
<class 'pandas.DataFrame'>
RangeIndex: 32436 entries, 0 to 32435
Data columns (total 15 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   station_eva_nr            32436 non-null  int64
 1   station_name              32436 non-null  str
 2   fahrt_id_part1            32436 non-null  str
 3   fahrt_id_part2            32436 non-null  str
 4   fahrt_id_part3            32436 non-null  int64
 5   train_category            32436 non-null  str
 6   line_ar                   29496 non-null  str
 7   line_dp                   29533 non-null  str
 8   train_no                  32436 non-null  str
 9   plan_arrival              29928 non-null  str
 10  arrival_platform          29928 non-null  str
 11  unmittelbarer_vorgaenger  29928 non-null  str
 12  plan_departure            29971 non-null  str
 13  departure_platform        29971 non-null  str
 14  unmittelbarer_nachfolger  29971 non-null  str
dtypes: int64(2), str(13)
memory usage: 6.9 MB
```

[11]:
```
passau = df_timetable_raw[df_timetable_raw.station_name == "Passau Hbf"]
```

[12]:
```
display(passau)
```

```
       station_eva_nr station_name        fahrt_id_part1 fahrt_id_part2  \
505           8000298   Passau Hbf   3825349029225220658     2609231203
506           8000298   Passau Hbf  -5145530745994233828     2609231248
507           8000298   Passau Hbf   -495216477104747713     2609231140
4690          8000298   Passau Hbf   6353612846196844485     2609231150
4691          8000298   Passau Hbf   7915775831881131053     2609231137
4692          8000298   Passau Hbf  -8369882101160635586     2609231336
9232          8000298   Passau Hbf   7926994611979689231     2609231250
9233          8000298   Passau Hbf  -8251015602825629679     2609231406
9234          8000298   Passau Hbf   3821279515324451281     2609231448
13529         8000298   Passau Hbf  -7103014634644358696     2609231350
13530         8000298   Passau Hbf   2465133487711427769     2609231540
13531         8000298   Passau Hbf  -7752766402398303321     2609231336
18023         8000298   Passau Hbf  -7526054729846747278     2609231540
```

```
18024         8000298   Passau Hbf  -6458831864484069596     2609231648
18025         8000298   Passau Hbf   5524195490450827193     2609231606
23003         8000298   Passau Hbf   4892006598205728564     2609231538
23004         8000298   Passau Hbf   1857208574100292543     2609231736
23005         8000298   Passau Hbf   3940389135840155310     2609231536
23006         8000298   Passau Hbf  -1658688830748151220     2609231550
28060         8000298   Passau Hbf    831784127940999986     2609231650
28061         8000298   Passau Hbf  -6616162589478058063     2609231840
28062         8000298   Passau Hbf   2511594504128930467     2609231806

       fahrt_id_part3 train_category line_ar line_dp train_no plan_arrival  \
505                 1             RB     NaN    RB46    27226          NaN
506                 1            REX     NaN      50     1781          NaN
507                10              R      50     NaN     5930   2609231225
4690               13            REX      50     NaN     1778   2609231312
4691               20             RB    RB46     NaN    27223   2609231353
4692                1              R     NaN      50     5935          NaN
9232               16              R      50     NaN     5908   2609231422
9233                1             RB     NaN    RB46    27230          NaN
9234                1            REX     NaN      50     1783          NaN
13529              13            REX      50     NaN     1780   2609231512
13530               1              R     NaN      50     5923          NaN
13531              21             RB    RB46     NaN    27229   2609231555
18023              10              R      50     NaN     5934   2609231625
18024               1            REX     NaN      50     1785          NaN
18025               1             RB     NaN    RB46    27234          NaN
23003              16              R      50     NaN     5910   2609231703
23004               1            REX     NaN      50     5937          NaN
23005              20             RB    RB46     NaN    27233   2609231755
23006              13            REX      50     NaN     1784   2609231712
28060              16              R      50     NaN     5912   2609231822
28061               1              R     NaN      50     5925          NaN
28062               1             RB     NaN    RB46    27238          NaN

      arrival_platform unmittelbarer_vorgaenger plan_departure  \
505                NaN                      NaN     2609231203
506                NaN                      NaN     2609231248
507                  3                Wernstein            NaN
4690                 5                Wernstein            NaN
4691                 5       Neustift(b Passau)            NaN
4692               NaN                      NaN     2609231336
9232                 3                Wernstein            NaN
9233               NaN                      NaN     2609231406
9234               NaN                      NaN     2609231448
13529                3                Wernstein            NaN
13530              NaN                      NaN     2609231540
13531                5       Neustift(b Passau)            NaN
18023                2                Wernstein            NaN
```

```
18024              NaN                      NaN     2609231648
18025              NaN                      NaN     2609231606
23003            5 A-C                Wernstein            NaN
23004              NaN                      NaN     2609231736
23005                5       Neustift(b Passau)            NaN
23006            6 A-C                Wernstein            NaN
28060                2                Wernstein            NaN
28061              NaN                      NaN     2609231840
28062              NaN                      NaN     2609231806

      departure_platform unmittelbarer_nachfolger
505                    5       Neustift(b Passau)
506                    3                Wernstein
507                  NaN                      NaN
4690                 NaN                      NaN
4691                 NaN                      NaN
4692                   5                Wernstein
9232                 NaN                      NaN
9233                   5       Neustift(b Passau)
9234                   3                Wernstein
13529                NaN                      NaN
13530                  3                Wernstein
13531                NaN                      NaN
18023                NaN                      NaN
18024                  2                Wernstein
18025                  5       Neustift(b Passau)
23003                NaN                      NaN
23004              5 A-C                Wernstein
23005                NaN                      NaN
23006                NaN                      NaN
28060                NaN                      NaN
28061                  6                Wernstein
28062                  5       Neustift(b Passau)
```

[13]:
```
df_timetable_raw.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 32436 entries, 0 to 32435
Data columns (total 15 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   station_eva_nr           32436 non-null  int64
 1   station_name             32436 non-null  str
 2   fahrt_id_part1           32436 non-null  str
 3   fahrt_id_part2           32436 non-null  str
 4   fahrt_id_part3           32436 non-null  int64
 5   train_category           32436 non-null  str
 6   line_ar                  29496 non-null  str
```

```
 7   line_dp                   29533 non-null  str
 8   train_no                  32436 non-null  str
 9   plan_arrival              29928 non-null  str
 10  arrival_platform          29928 non-null  str
 11  unmittelbarer_vorgaenger  29928 non-null  str
 12  plan_departure            29971 non-null  str
 13  departure_platform        29971 non-null  str
 14  unmittelbarer_nachfolger  29971 non-null  str
dtypes: int64(2), str(13)
memory usage: 6.9 MB
```

1.8 Aufbereitung der PySpark Umgebung

Die Bearbeitung der Informationen und Aufbereitung eines Graph konformen Ergebnisses wird mit PySpark durchgeführt.

[14]:
```
# 1. Systemkonforme Java-Umgebung aktivieren (auf dem remote PC)
user_home = os.path.expanduser("~")
os.environ["JAVA_HOME"] = str(Path(user_home) / ".jdk" / "jdk-17.0.20.1+1")
os.environ["PATH"] = os.environ["JAVA_HOME"] + "\\bin;" + os.environ.
 ↪get("PATH", "")

# Initialisierung lokaler SparkSession
spark = SparkSession.builder \
    .master("local[*]") \
    .appName("Phase4A_Fahrplan_Graph_Preparation") \
    .config("spark.sql.warehouse.dir", os.path.join(user_home,␣
 ↪"spark-warehouse")) \
    .config("spark.local.dir", os.path.join(user_home, "spark-scratch")) \
    .config("spark.ui.port", "4055") \
    .getOrCreate()

# 2. Explizite Schema-Definition für die fehlerfreie JVM-Konvertierung des␣
 ↪Fahrplan-Caches
timetable_schema = StructType([
    StructField("station_eva_nr", LongType(), True),
    StructField("station_name", StringType(), True),
    StructField("fahrt_id_part1", StringType(), True),
    StructField("fahrt_id_part2", StringType(), True),
    StructField("fahrt_id_part3", IntegerType(), True),
    StructField("train_category", StringType(), True),
    StructField("line_ar", StringType(), True),
    StructField("line_dp", StringType(), True),
    StructField("train_no", StringType(), True),
    StructField("plan_arrival", StringType(), True),
    StructField("arrival_platform", StringType(), True),
    StructField("unmittelbarer_vorgaenger", StringType(), True),
    StructField("plan_departure", StringType(), True),
```

```
    StructField("departure_platform", StringType(), True),
    StructField("unmittelbarer_nachfolger", StringType(), True)
])

# NaN/NaT-Werte im Pandas-DataFrame vor der Konvertierung in saubere␣
 ↪Python-None-Typen transformieren
df_timetable_clean_types = df_timetable_raw.astype(object).where(pd.
 ↪notnull(df_timetable_raw), None)

df_spark_base = spark.createDataFrame(df_timetable_clean_types,␣
 ↪schema=timetable_schema)

# Registrierung der zwingend erforderlichen Quell-Sicht
df_spark_base.createOrReplaceTempView("view_p4_rohdaten")
print(f"Erfolgreich konvertiert! Sicht 'view_p4_rohdaten' registriert. Zeilen:␣
 ↪{df_spark_base.count()}")

# ========================================================================
# ZUSATZ-INGESTION: StaDa-Referenzdaten parallel für den ID-Lookup laden
# ========================================================================
print(f"Lade offizielle DB-Referenzdaten von:␣
 ↪{get_project_relative_path(STADA_API_RESULTS)}")
df_stada_ref = pd.read_parquet(STADA_API_RESULTS)

# Konvertierung und Bereinigung für Spark
df_stada_clean_types = df_stada_ref.astype(object).where(pd.
 ↪notnull(df_stada_ref), None)
df_spark_stada = spark.createDataFrame(df_stada_clean_types)

# Registrierung der Referenz-Sicht für spätere Joins
df_spark_stada.createOrReplaceTempView("view_p4_stada_referenz")
print(f"Erfolgreich! Referenz-Sicht 'view_p4_stada_referenz' im SQL-Namensraum␣
 ↪fixiert.")
```

```
C:\0_DA\Python312\Lib\site-packages\pyspark\testing\utils.py:127: FutureWarning:
PySpark does not yet fully support pandas >= 3.0.0. Some features may not work
correctly. It is recommended to use pandas < 3.0.0 for now.
  require_minimum_pandas_version()
C:\0_DA\Python312\Lib\site-packages\pyspark\sql\pandas\conversion.py:659:
FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some features
may not work correctly. It is recommended to use pandas < 3.0.0 for now.
  require_minimum_pandas_version()
C:\0_DA\Python312\Lib\site-packages\pyspark\sql\pandas\conversion.py:936:
FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some features
may not work correctly. It is recommended to use pandas < 3.0.0 for now.
  require_minimum_pandas_version()
```

```
Erfolgreich konvertiert! Sicht 'view_p4_rohdaten' registriert. Zeilen: 32436
```

```
Lade offizielle DB-Referenzdaten von: outputs/04_results.parquet
Erfolgreich! Referenz-Sicht 'view_p4_stada_referenz' im SQL-Namensraum fixiert.

C:\0_DA\Python312\Lib\site-packages\pyspark\sql\pandas\conversion.py:659:
FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some features
may not work correctly. It is recommended to use pandas < 3.0.0 for now.
  require_minimum_pandas_version()
C:\0_DA\Python312\Lib\site-packages\pyspark\sql\pandas\conversion.py:936:
FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some features
may not work correctly. It is recommended to use pandas < 3.0.0 for now.
  require_minimum_pandas_version()
```

[15]:
```
# UNTERSUCHUNG VORGANG: Schrittweise Isolation aller realen Netzknoten
# Gruppiert nach der EVA-Nummer und dem Namen, um Duplikate zu entfernen
spark.sql("""
    CREATE OR REPLACE TEMPORARY VIEW view_p4_eindeutige_knoten AS
    SELECT DISTINCT
        station_eva_nr AS knoten_eva_nr,
        station_name AS knoten_station_name
    FROM view_p4_rohdaten
    WHERE station_eva_nr IS NOT NULL
""")

print("--- Struktur der isolierten Knoten-Sicht ---")
spark.sql("DESCRIBE view_p4_eindeutige_knoten").show(truncate=False)

# Eindeutige Paare im Fahrplan-Cache
anzahl_knoten_basis = spark.sql("SELECT COUNT(*) FROM␣
 ↪view_p4_eindeutige_knoten").collect()[0][0]
print(f"Empirischer Zwischenstand: {anzahl_knoten_basis} eindeutige␣
 ↪EVA-Namens-Paare im Fahrplan-Cache identifiziert.")

print("\n--- Auszug der ersten Knoten-Paare ---")
spark.sql("""
    SELECT * FROM view_p4_eindeutige_knoten
    ORDER BY knoten_station_name
    LIMIT 10
""").show(truncate=False)
```

```
--- Struktur der isolierten Knoten-Sicht ---
+------------------+---------+-------+
|col_name          |data_type|comment|
+------------------+---------+-------+
|knoten_eva_nr     |bigint   |NULL   |
|knoten_station_name|string   |NULL   |
+------------------+---------+-------+

Empirischer Zwischenstand: 889 eindeutige EVA-Namens-Paare im Fahrplan-Cache
```

```
identifiziert.

--- Auszug der ersten Knoten-Paare ---
+-------------+-------------------+
|knoten_eva_nr|knoten_station_name|
+-------------+-------------------+
|8000410      |Abensberg          |
|8000419      |Adelschlag         |
|8000420      |Adelsdorf(Mittelfr)|
|8000433      |Agatharied         |
|8000452      |Aichach            |
|8000459      |Ainring            |
|8000482      |Aletshausen        |
|8000498      |Allersberg(Rothsee)|
|8000504      |Altdorf West       |
|8000509      |Altdorf(b Nürnberg)|
+-------------+-------------------+
```

[16]:
```
# drei weitere München Hbf Namen
spark.sql("""
    SELECT *
    FROM view_p4_eindeutige_knoten
    WHERE knoten_eva_nr > 8098260
""").show(truncate=False)
```

```
+-------------+--------------------+
|knoten_eva_nr|knoten_station_name  |
+-------------+--------------------+
|8098261      |München Hbf Gl.27-36|
|8098262      |München Hbf Gl.5-10 |
|8098263      |München Hbf (tief)  |
+-------------+--------------------+
```

[17]:
```
print("Erweitere das Fahrplan-Grundgerüst um relationale ID-Zielspalten...")

# .withColumn() fügt die Spalten hinzu und füllt sie initial mit einem leeren␣
  ↪SQL-Nullwert (None)
df_spark_extended = df_spark_base \
    .withColumn("vorgaenger_eva_nr", F.lit(None).cast(LongType())) \
    .withColumn("nachfolger_eva_nr", F.lit(None).cast(LongType()))

# Registrierung der erweiterten Tabelle als neue, saubere Quell-Ansicht
df_spark_extended.createOrReplaceTempView("view_p4_master_erweitert")

print("--- Struktur der erweiterten Master-Tabelle (Inklusive neuer ID-Spalten)␣
  ↪---")
```

```
spark.sql("DESCRIBE view_p4_master_erweitert").show(truncate=False)
```

```
Erweitere das Fahrplan-Grundgerüst um relationale ID-Zielspalten…
--- Struktur der erweiterten Master-Tabelle (Inklusive neuer ID-Spalten) ---
+------------------------+---------+-------+
|col_name                |data_type|comment|
+------------------------+---------+-------+
|station_eva_nr          |bigint   |NULL   |
|station_name            |string   |NULL   |
|fahrt_id_part1          |string   |NULL   |
|fahrt_id_part2          |string   |NULL   |
|fahrt_id_part3          |int      |NULL   |
|train_category          |string   |NULL   |
|line_ar                 |string   |NULL   |
|line_dp                 |string   |NULL   |
|train_no                |string   |NULL   |
|plan_arrival            |string   |NULL   |
|arrival_platform        |string   |NULL   |
|unmittelbarer_vorgaenger|string   |NULL   |
|plan_departure          |string   |NULL   |
|departure_platform      |string   |NULL   |
|unmittelbarer_nachfolger|string   |NULL   |
|vorgaenger_eva_nr       |bigint   |NULL   |
|nachfolger_eva_nr       |bigint   |NULL   |
+------------------------+---------+-------+
```

[18]:
```
# UNTERSUCHUNG VORGANG: Kombinierte ID- und Namens-Konsolidierung der Netzkanten
print("Berechne und befülle die neuen ID-Zielspalten im Fahrplan-Skelett...")

df_consolidated_master = spark.sql("""
    WITH cte_window_sequencing AS (
        SELECT
            m.*,
            -- Weg A: Mathematische Ermittlung des Vorgängers/Nachfolgers über␣
 ↪die Zug-ID (Fensterfunktion)
            LAG(station_eva_nr) OVER (
                PARTITION BY fahrt_id_part1, fahrt_id_part2
                ORDER BY fahrt_id_part3
            ) AS window_vorgaenger_eva,

            LEAD(station_eva_nr) OVER (
                PARTITION BY fahrt_id_part1, fahrt_id_part2
                ORDER BY fahrt_id_part3
            ) AS window_nachfolger_eva
        FROM view_p4_master_erweitert m
    )
```

```
    SELECT
        w.station_eva_nr,
        w.station_name,
        w.fahrt_id_part1,
        w.fahrt_id_part2,
        w.fahrt_id_part3,
        w.train_category,
        w.line_ar,
        w.line_dp,
        w.train_no,
        to_timestamp(w.plan_arrival, 'yyMMddHHmm') AS plan_arrival_dt,
        w.arrival_platform,
        w.unmittelbarer_vorgaenger,
        to_timestamp(w.plan_departure, 'yyMMddHHmm') AS plan_departure_dt,
        w.departure_platform,
        w.unmittelbarer_nachfolger,

        -- Befüllen der Vorgänger-ID: Erst über die Zug-ID, sonst über das␣
 ↪Namens-Sicherheitsnetz
        COALESCE(
            w.window_vorgaenger_eva,
            ref_v.api_eva_nr
        ) AS vorgaenger_eva_nr,

        -- Befüllen der Nachfolger-ID: Erst über die Zug-ID, sonst über das␣
 ↪Namens-Sicherheitsnetz
        COALESCE(
            w.window_nachfolger_eva,
            ref_n.api_eva_nr
        ) AS nachfolger_eva_nr

    FROM cte_window_sequencing w
    LEFT JOIN view_p4_stada_referenz ref_v ON w.unmittelbarer_vorgaenger =␣
 ↪ref_v.api_station_name
    LEFT JOIN view_p4_stada_referenz ref_n ON w.unmittelbarer_nachfolger =␣
 ↪ref_n.api_station_name
""")

# Registrierung der vollkommen bereinigten Gesamt-Matrix als SQL-Sicht
df_consolidated_master.createOrReplaceTempView("view_p4_master_konsolidiert")
print("Sicht 'view_p4_master_konsolidiert' erfolgreich im relationalen System␣
 ↪verankert.")
```

```
Berechne und befülle die neuen ID-Zielspalten im Fahrplan-Skelett…
Sicht 'view_p4_master_konsolidiert' erfolgreich im relationalen System
verankert.
```

[19]:
```
print("--- QUALITÄTSKONTROLLE: Überprüfung der neu befüllten ID-Spalten ---")

spark.sql("""
    SELECT
        station_eva_nr,
        station_name,
        unmittelbarer_vorgaenger,
        cast(vorgaenger_eva_nr as bigint) AS vorgaenger_eva_nr,
        unmittelbarer_nachfolger,
        cast(nachfolger_eva_nr as bigint) AS nachfolger_eva_nr
    FROM view_p4_master_konsolidiert
    WHERE station_name LIKE 'München Hbf%' OR station_name = 'Passau Hbf'
    LIMIT 15
""").show(truncate=False)
```

```
--- QUALITÄTSKONTROLLE: Überprüfung der neu befüllten ID-Spalten ---
+-------------+--------------------+--------------------------+---------------
--+--------------------------+-----------------+
|station_eva_nr|station_name        |unmittelbarer_vorgaenger
|vorgaenger_eva_nr|unmittelbarer_nachfolger   |nachfolger_eva_nr|
+-------------+--------------------+--------------------------+---------------
--+--------------------------+-----------------+
|8098263       |München Hbf (tief)  |München Karlsplatz         |8004132
|München Hackerbrücke       |8004129          |
|8098263       |München Hbf (tief)  |München Karlsplatz         |8004132
|München Hackerbrücke       |8004129          |
|8000261       |München Hbf         |NULL                       |NULL
|Ingolstadt Hbf             |8000183          |
|8098261       |München Hbf Gl.27-36|München Donnersbergerbrücke|8004128
|NULL                       |NULL             |
|8000261       |München Hbf         |München-Pasing             |8004158
|NULL                       |NULL             |
|8098263       |München Hbf (tief)  |München Karlsplatz         |8004132
|München Hackerbrücke       |8004129          |
|8098263       |München Hbf (tief)  |München Karlsplatz         |8004132
|München Hackerbrücke       |8004129          |
|8098263       |München Hbf (tief)  |München Hackerbrücke       |8004129
|München Karlsplatz         |8004132          |
|8098261       |München Hbf Gl.27-36|NULL                       |NULL
|München Donnersbergerbrücke|8004128          |
|8098263       |München Hbf (tief)  |München Karlsplatz         |8004132
|München Hackerbrücke       |8004129          |
|8098261       |München Hbf Gl.27-36|NULL                       |NULL
|München Donnersbergerbrücke|8004128          |
|8098263       |München Hbf (tief)  |München Karlsplatz         |8004132
|München Hackerbrücke       |8004129          |
|8098261       |München Hbf Gl.27-36|München Donnersbergerbrücke|8004128
|NULL                       |NULL             |
```

```
|8098261       |München Hbf Gl.27-36|NULL                        |NULL
|München Donnersbergerbrücke|8004128          |
|8000298       |Passau Hbf          |Wernstein                   |NULL
|NULL                        |NULL             |
+--------------+--------------------+----------------------------+--------------
--+----------------------------+----------------+
```

[20]:
```
# UNTERSUCHUNG VORGANG: Prozentuale Quantifizierung der nicht-aufgelösten␣
 ↪Netzkanten-IDs
print("--- Prozentuale Fehlteil-Analyse der berechneten ID-Spalten ---")

spark.sql("""
    SELECT
        -- Metriken für den Vorgänger
        COUNT(unmittelbarer_vorgaenger) AS gesamt_text_vorgaenger,
        SUM(CASE WHEN vorgaenger_eva_nr IS NULL THEN 1 ELSE 0 END) AS␣
 ↪vorgaenger_fehlend_anzahl,
        ROUND((SUM(CASE WHEN vorgaenger_eva_nr IS NULL THEN 1 ELSE 0 END) /␣
 ↪COUNT(unmittelbarer_vorgaenger)) * 100, 2) AS vorgaenger_fehlquote_prozent,

        -- Metriken für den Nachfolger
        COUNT(unmittelbarer_nachfolger) AS gesamt_text_nachfolger,
        SUM(CASE WHEN nachfolger_eva_nr IS NULL THEN 1 ELSE 0 END) AS␣
 ↪nachfolger_fehlend_anzahl,
        ROUND((SUM(CASE WHEN nachfolger_eva_nr IS NULL THEN 1 ELSE 0 END) /␣
 ↪COUNT(unmittelbarer_nachfolger)) * 100, 2) AS nachfolger_fehlquote_prozent

    FROM view_p4_master_konsolidiert
""").show(truncate=False)
```

```
--- Prozentuale Fehlteil-Analyse der berechneten ID-Spalten ---
+---------------------+------------------------+---------------------------+-
--------------------+------------------------+---------------------------+
|gesamt_text_vorgaenger|vorgaenger_fehlend_anzahl|vorgaenger_fehlquote_prozent|g
esamt_text_nachfolger|nachfolger_fehlend_anzahl|nachfolger_fehlquote_prozent|
+---------------------+------------------------+---------------------------+-
--------------------+------------------------+---------------------------+
|30654                |2839                    |9.26                       |
|30694                |2818                    |9.18                       |
+---------------------+------------------------+---------------------------+-
--------------------+------------------------+---------------------------+
```

[21]:
```
# UNTERSUCHUNG VORGANG: Stichpunktartige Recherche zur Identifikation␣
 ↪abweichender Stationsbezeichnungen
```

```
print("--- Auszug aller registrierten Stationsnamen mit dem Fragment␣
 ↪'Neustift(b Passau)' ---")

spark.sql("""
    SELECT DISTINCT
        knoten_eva_nr,
        knoten_station_name
    FROM view_p4_eindeutige_knoten
    WHERE knoten_station_name LIKE '%Passau%'
     OR knoten_station_name LIKE '%Neustift%'
    ORDER BY knoten_station_name ASC
""").show(truncate=False)
```

```
--- Auszug aller registrierten Stationsnamen mit dem Fragment 'Neustift(b
Passau)' ---
+-------------+-------------------+
|knoten_eva_nr|knoten_station_name|
+-------------+-------------------+
|8000298      |Passau Hbf         |
+-------------+-------------------+
```

[22]:
```
# UNTERSUCHUNG VORGANG: Sequenz-Verfolgung des Passauer Umlaufs über die␣
 ↪ID-Parts
print("--- Chronologische Reise-Analyse für Zug-Umlauf: 3825349029225220658␣
 ↪---")

spark.sql("""
    SELECT
        station_eva_nr,
        station_name,
        fahrt_id_part1,
        fahrt_id_part2,
        fahrt_id_part3,
        train_category,
        train_no,
        line_dp,
        line_ar,
        plan_arrival_dt,
        plan_departure_dt,
        unmittelbarer_vorgaenger,
        unmittelbarer_nachfolger,
        cast(vorgaenger_eva_nr as bigint) AS vorgaenger_eva_nr,
        cast(nachfolger_eva_nr as bigint) AS nachfolger_eva_nr
    FROM view_p4_master_konsolidiert
    WHERE fahrt_id_part1 = '3825349029225220658'
      AND fahrt_id_part2 = '2609231203'
```

```
    ORDER BY fahrt_id_part3 ASC
""").show(truncate=False)
```

```
--- Chronologische Reise-Analyse für Zug-Umlauf: 3825349029225220658 ---
+--------------+------------+------------------+--------------+--------------+-
------------+--------+-------+-------+---------------+------------------+-----
------------------+-----------------------+-----------------+----------------
+
|station_eva_nr|station_name|fahrt_id_part1     |fahrt_id_part2|fahrt_id_part3|t
rain_category|train_no|line_dp|line_ar|plan_arrival_dt|plan_departure_dt  |unmit
telbarer_vorgaenger|unmittelbarer_nachfolger|vorgaenger_eva_nr|nachfolger_eva_nr
|
+--------------+------------+------------------+--------------+--------------+-
------------+--------+-------+-------+---------------+------------------+-----
------------------+-----------------------+-----------------+----------------
+
|8000298       |Passau Hbf  |3825349029225220658|2609231203    |1
|RB            |27226   |RB46   |NULL   |NULL           |2026-09-23
12:03:00|NULL                    |Neustift(b Passau)      |NULL
|NULL             |
+--------------+------------+------------------+--------------+--------------+-
------------+--------+-------+-------+---------------+------------------+-----
------------------+-----------------------+-----------------+----------------
+
```

1.8.1 Datenkritischer Befund zum systemischen Datenverlust (Beispiel Passau Hbf)

Ergebnis: Die empirische Stichprobenanalyse des Zugumlaufs `RB46` offenbart ein schwerwiegendes Defizit in der Datenkonsistenz der DB-API. Obwohl der Zug einen rein innerbayerischen Streckenverlauf (Passau Hbf Richtung Mühldorf) besitzt, enthält der gesamte extrahierte 6-Stunden-Fahrplancache außer dem Startbahnhof Passau Hbf keinerlei weitere Stationen oder IDs zu dieser Fahrt.

Dies liefert den eindeutigen informationstechnischen Nachweis, dass bestimmte Zugläufe im operativen API-Datenstrom der Deutschen Bahn unvollständig übertragen werden und somit netzweit nicht mehr nachvollziehbar sind.

[23]:
```
# UNTERSUCHUNG VORGANG: Sequenzielle Kantenaufdeckung über exaktes␣
 ↪Takt-Inkrement (part3 + 1)
print("--- Generiere sequenzielle Nachfolger-Sicht für unvollständige ID-Knoten␣
 ↪---")

spark.sql("""
    CREATE OR REPLACE TEMPORARY VIEW view_p4_nachfolger_inkrement AS
    SELECT
        aktuell.station_eva_nr AS quell_eva_nr,
        aktuell.station_name AS quell_station_name,
```

```
        folge.station_eva_nr AS berechnete_ziel_eva_nr,
        folge.station_name AS berechnete_ziel_station_name,
        aktuell.fahrt_id_part1,
        aktuell.fahrt_id_part2,
        aktuell.fahrt_id_part3 AS quell_part3,
        folge.fahrt_id_part3 AS ziel_part3,
        aktuell.train_category,
        aktuell.train_no,
        COALESCE(aktuell.line_dp, aktuell.line_ar, aktuell.train_category) AS␣
 ↪zug_linie,
        aktuell.plan_departure_dt AS abfahrtszeit_quelle
    FROM view_p4_master_konsolidiert aktuell
    INNER JOIN view_p4_master_konsolidiert folge ON
        aktuell.fahrt_id_part1 = folge.fahrt_id_part1
        AND aktuell.fahrt_id_part2 = folge.fahrt_id_part2
        AND folge.fahrt_id_part3 = aktuell.fahrt_id_part3 + 1  -- Strikte␣
 ↪Bedingung: Genau die nächste Station
    WHERE aktuell.nachfolger_eva_nr IS NULL                    -- Nur wo die␣
 ↪ID bisher gefehlt hat
""")

print("Sicht 'view_p4_nachfolger_inkrement' erfolgreich im SQL-Namensraum␣
 ↪registriert.\n")

# --- QUALITÄTSKONTROLLE UND COUNT-CHECK ---
print("--- Auszug der über das part3-Inkrement neu aufgedeckten Netzkanten ---")
df_check_inkrement = spark.sql("""
    SELECT
        fahrt_id_part1,
        quell_part3,
        quell_station_name,
        cast(quell_eva_nr as bigint) AS quell_eva_nr,
        berechnete_ziel_station_name,
        cast(berechnete_ziel_eva_nr as bigint) AS ziel_eva_nr,
        zug_linie
    FROM view_p4_nachfolger_inkrement
    ORDER BY fahrt_id_part1 ASC, quell_part3 ASC
    LIMIT 15
""")

df_check_inkrement.show(truncate=False)

# Absolute Zählung der neu gewonnenen Kanten für die Dokumentation
anzahl_neue_kanten = spark.sql("SELECT COUNT(*) FROM␣
 ↪view_p4_nachfolger_inkrement").collect()[0][0]
print(f"Statistischer Befund: Über das exakte part3-Inkrement (+1) wurden␣
 ↪{anzahl_neue_kanten} zusätzliche Kanten rekonstruiert.")
```

```
--- Generiere sequenzielle Nachfolger-Sicht für unvollständige ID-Knoten ---
Sicht 'view_p4_nachfolger_inkrement' erfolgreich im SQL-Namensraum registriert.

--- Auszug der über das part3-Inkrement neu aufgedeckten Netzkanten ---
+--------------+-----------+-----------------+------------+--------------------
--------+-----------+---------+
|fahrt_id_part1|quell_part3|quell_station_name|quell_eva_nr|berechnete_ziel_stat
ion_name|ziel_eva_nr|zug_linie|
+--------------+-----------+-----------------+------------+--------------------
--------+-----------+---------+
+--------------+-----------+-----------------+------------+--------------------
--------+-----------+---------+

Statistischer Befund: Über das exakte part3-Inkrement (+1) wurden 0 zusätzliche
Kanten rekonstruiert.
```

[24]:
```
# UNTERSUCHUNG VORGANG: Sequenzielle Kantenaufdeckung über exaktes␣
 ↪Takt-Dekrement (part3 - 1)
print("--- Generiere sequenzielle Vorgänger-Sicht für unvollständige ID-Knoten␣
 ↪---")

spark.sql("""
    CREATE OR REPLACE TEMPORARY VIEW view_p4_vorgaenger_dekrement AS
    SELECT
        vorgaenger.station_eva_nr AS berechnete_quell_eva_nr,
        vorgaenger.station_name AS berechnete_quell_station_name,
        aktuell.station_eva_nr AS ziel_eva_nr,
        aktuell.station_name AS ziel_station_name,
        aktuell.fahrt_id_part1,
        aktuell.fahrt_id_part2,
        vorgaenger.fahrt_id_part3 AS quell_part3,
        aktuell.fahrt_id_part3 AS ziel_part3,
        aktuell.train_category,
        aktuell.train_no,
        COALESCE(aktuell.line_ar, aktuell.line_dp, aktuell.train_category) AS␣
 ↪zug_linie,
        aktuell.plan_arrival_dt AS ankunftszeit_ziel
    FROM view_p4_master_konsolidiert aktuell
    INNER JOIN view_p4_master_konsolidiert vorgaenger ON
        aktuell.fahrt_id_part1 = vorgaenger.fahrt_id_part1
        AND aktuell.fahrt_id_part2 = vorgaenger.fahrt_id_part2
        AND vorgaenger.fahrt_id_part3 = aktuell.fahrt_id_part3 - 1  -- Strikte␣
 ↪Bedingung: Genau die vorherige Station
    WHERE aktuell.vorgaenger_eva_nr IS NULL                         -- Nur wo␣
 ↪die ID bisher gefehlt hat
""")
```

```
print("Sicht 'view_p4_vorgaenger_dekrement' erfolgreich im SQL-Namensraum␣
 ↪registriert.\n")

# --- QUALITÄTSKONTROLLE UND COUNT-CHECK ---
print("--- Auszug der über das part3-Dekrement neu aufgedeckten Netzkanten ---")
df_check_dekrement = spark.sql("""
    SELECT
        fahrt_id_part1,
        quell_part3,
        berechnete_quell_station_name AS quell_station_name,
        cast(berechnete_quell_eva_nr as bigint) AS quell_eva_nr,
        ziel_station_name,
        cast(ziel_eva_nr as bigint) AS ziel_eva_nr,
        zug_linie
    FROM view_p4_vorgaenger_dekrement
    ORDER BY fahrt_id_part1 ASC, quell_part3 ASC
    LIMIT 15
""")

df_check_dekrement.show(truncate=False)

# Absolute Zählung der neu gewonnenen Kanten für die Dokumentation
anzahl_neue_vorgaenger = spark.sql("SELECT COUNT(*) FROM␣
 ↪view_p4_vorgaenger_dekrement").collect()[0][0]
print(f"Statistischer Befund: Über das exakte part3-Dekrement (-1) wurden␣
 ↪{anzahl_neue_vorgaenger} zusätzliche Kanten rekonstruiert.")
```

```
--- Generiere sequenzielle Vorgänger-Sicht für unvollständige ID-Knoten ---
Sicht 'view_p4_vorgaenger_dekrement' erfolgreich im SQL-Namensraum registriert.

--- Auszug der über das part3-Dekrement neu aufgedeckten Netzkanten ---
+--------------+-----------+-----------------+------------+-----------------+--
---------+---------+
|fahrt_id_part1|quell_part3|quell_station_name|quell_eva_nr|ziel_station_name|zi
el_eva_nr|zug_linie|
+--------------+-----------+-----------------+------------+-----------------+--
---------+---------+
+--------------+-----------+-----------------+------------+-----------------+--
---------+---------+

Statistischer Befund: Über das exakte part3-Dekrement (-1) wurden 0 zusätzliche
Kanten rekonstruiert.
```

[25]:
```
# UNTERSUCHUNG VORGANG: Berechnung der bereinigten, wahren Fehlquote für den␣
 ↪Vorgänger
print("--- Bereinigte Fehlteil-Analyse für den Vorgänger (Nur Zwischenhalte mit␣
 ↪part3 > 1) ---")
```

```
print("--- Ausschluss struktureller Startstationen: (part3 == 1) ->␣
 ↪Startbahnhof ---")

spark.sql("""
    SELECT
        -- Anzahl aller echten Zwischen- und Endhaltestellen
        COUNT(CASE WHEN fahrt_id_part3 > 1 THEN 1 END) AS␣
 ↪gesamt_relevante_zwischenhalte,

        -- Anzahl jener Zwischenhaltestellen, bei denen die Vorgänger-ID␣
 ↪TROTZDEM fehlt
        SUM(CASE WHEN fahrt_id_part3 > 1 AND vorgaenger_eva_nr IS NULL THEN 1␣
 ↪ELSE 0 END) AS echte_fehlteile_anzahl,

        -- Die bereinigte, wahre Fehlquote in Prozent
        ROUND(
            (SUM(CASE WHEN fahrt_id_part3 > 1 AND vorgaenger_eva_nr IS NULL␣
 ↪THEN 1 ELSE 0 END) /
             COUNT(CASE WHEN fahrt_id_part3 > 1 THEN 1 END)) * 100, 2
        ) AS wahre_vorgaenger_fehlquote_prozent

    FROM view_p4_master_konsolidiert
""").show(truncate=False)
```

```
--- Bereinigte Fehlteil-Analyse für den Vorgänger (Nur Zwischenhalte mit part3 >
1) ---
--- Ausschluss struktureller Startstationen: (part3 == 1) -> Startbahnhof ---
+------------------------------+----------------------+-------------------------
---------+
|gesamt_relevante_zwischenhalte|echte_fehlteile_anzahl|wahre_vorgaenger_fehlquot
e_prozent|
+------------------------------+----------------------+-------------------------
---------+
|30654                         |331                   |1.08
|
+------------------------------+----------------------+-------------------------
---------+
```

[26]:
```
# UNTERSUCHUNG VORGANG: Exakte Quantifizierung des Nachfolger-Datenverlusts auf␣
 ↪Basis der XML-Textpfade
print("--- KORRIGIERTE Fehlteil-Analyse für den Nachfolger (Ausschluss␣
 ↪struktureller Endstationen) ---")

spark.sql("""
    SELECT
```

```
        -- Anzahl aller Zeilen, bei denen laut XML-Fahrplan ein␣
 ↪Nachfolgebahnhof existiert
        COUNT(unmittelbarer_nachfolger) AS gesamt_relevante_fahrten,

        -- Anzahl jener Fahrten, bei denen der Name existiert, aber keine␣
 ↪EVA-ID ermittelt wurde (Grenzverkehr/Lücken)
        SUM(CASE WHEN unmittelbarer_nachfolger IS NOT NULL AND␣
 ↪nachfolger_eva_nr IS NULL THEN 1 ELSE 0 END) AS echte_nachfolger_fehlteile,

        -- Die korrekte, physische Fehlquote für den Nachfolger in Prozent
        ROUND(
            (SUM(CASE WHEN unmittelbarer_nachfolger IS NOT NULL AND␣
 ↪nachfolger_eva_nr IS NULL THEN 1 ELSE 0 END) /
             COUNT(unmittelbarer_nachfolger)) * 100, 2
        ) AS wahre_nachfolger_fehlquote_prozent

    FROM view_p4_master_konsolidiert
""").show(truncate=False)
```

```
--- KORRIGIERTE Fehlteil-Analyse für den Nachfolger (Ausschluss struktureller
Endstationen) ---
+------------------------+-------------------------+--------------------------
-------+
|gesamt_relevante_fahrten|echte_nachfolger_fehlteile|wahre_nachfolger_fehlquote_
prozent|
+------------------------+-------------------------+--------------------------
-------+
|30694                   |350                      |1.14
|
+------------------------+-------------------------+--------------------------
-------+
```

1.8.2 Evaluierung der verbleibenden Systemlücken

Die verbleibende Rest-Fehlquote von knapp über 1 % ist analytisch präzise begründet: Sie resultiert ausschließlich aus transienten Zugläufen über die Bundesland- und Landesgrenzen hinweg sowie aus unvollständigen Fahrplan-Fragmenten an den äußeren Netzrändern.

[27]:
```
print("=" * 70)
print("FUSIONIERUNG: Kombiniere alle validierten Zu- und Ablaufkanten...")
print("=" * 70)

# UNION ALL aggregiert alle physisch existierenden Kanten (Vorgänger -> Aktuell␣
 ↪und Aktuell -> Nachfolger)
df_final_spark_edges = spark.sql("""
    WITH cte_alle_kanten AS (
        -- 1. Teil: Alle validen Ablauf-Verbindungen
```

```
        SELECT
            fahrt_id_part1, fahrt_id_part2, fahrt_id_part3, train_category,␣
 ↪train_no,
            COALESCE(line_dp, line_ar, train_category) AS zug_linie,
            station_eva_nr AS quell_eva_nr,
            station_name AS quell_station_name,
            plan_departure_dt AS abfahrtszeit_quelle,
            cast(nachfolger_eva_nr as bigint) AS ziel_eva_nr,
            unmittelbarer_nachfolger AS ziel_station_name
        FROM view_p4_master_konsolidiert
        WHERE nachfolger_eva_nr IS NOT NULL

        UNION ALL

        -- 2. Teil: Alle validen Zulauf-Verbindungen
        SELECT
            fahrt_id_part1, fahrt_id_part2, fahrt_id_part3, train_category,␣
 ↪train_no,
            COALESCE(line_ar, line_dp, train_category) AS zug_linie,
            cast(vorgaenger_eva_nr as bigint) AS quell_eva_nr,
            unmittelbarer_vorgaenger AS quell_station_name,
            plan_arrival_dt AS abfahrtszeit_quelle,
            station_eva_nr AS ziel_eva_nr,
            station_name AS ziel_station_name
        FROM view_p4_master_konsolidiert
        WHERE vorgaenger_eva_nr IS NOT NULL
    )
    SELECT DISTINCT
        fahrt_id_part1,
        fahrt_id_part2,
        fahrt_id_part3,
        train_category,
        zug_linie,
        train_no,
        quell_eva_nr,
        quell_station_name,
        abfahrtszeit_quelle,
        ziel_eva_nr,
        ziel_station_name
    FROM cte_alle_kanten
    WHERE quell_eva_nr != ziel_eva_nr -- Verhindert zirkuläre Eigenschleifen
    ORDER BY fahrt_id_part1, fahrt_id_part3
""")

print("\nÜberführe das hochpräzise Kantenmodell via toPandas in den lokalen␣
 ↪Speicher...")
pdf_graph_final = df_final_spark_edges.toPandas()
```

```python
# Absicherung des Export-Pfads
EXPORT_PATH_STRING = get_project_relative_path(TIMETABLE_API_RESULTS)
physischer_pfad_p4 = Path(EXPORT_PATH_STRING)

if physischer_pfad_p4.exists():
    if physischer_pfad_p4.is_dir():
        shutil.rmtree(physischer_pfad_p4)
    else:
        physischer_pfad_p4.unlink()

print(f"Sichere fertige Kanten- und Zuginformationsbasis unter:␣
 ↪{EXPORT_PATH_STRING}")
pdf_graph_final.to_parquet(EXPORT_PATH_STRING, index=False)

# Ordnungsgemäßes Herunterfahren der JVM-Ressourcen
spark.stop()

print("\n" + "=" * 70)
print(f"Dimensionen der exportierten Master-Kantenliste (Shape):␣
 ↪{pdf_graph_final.shape}")
print("=" * 70)
```

```
======================================================================
FUSIONIERUNG: Kombiniere alle validierten Zu- und Ablaufkanten…
======================================================================

Überführe das hochpräzise Kantenmodell via toPandas in den lokalen Speicher…

C:\0_DA\Python312\Lib\site-packages\pyspark\sql\pandas\conversion.py:298:
FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some features
may not work correctly. It is recommended to use pandas < 3.0.0 for now.
  require_minimum_pandas_version()
C:\0_DA\Python312\Lib\site-packages\pyspark\sql\pandas\types.py:778:
FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some features
may not work correctly. It is recommended to use pandas < 3.0.0 for now.
  require_minimum_pandas_version()

Sichere fertige Kanten- und Zuginformationsbasis unter:
outputs/05_results.parquet

======================================================================
Dimensionen der exportierten Master-Kantenliste (Shape): (58268, 11)
======================================================================
```

2 Dokumentation des Systemzustands

Dieses Notebook dokumentiert den operativen Entwicklungs- und Forschungsstand der Phase 4A. Der Code ist explizit als lauffähiger Prototyp für iterative Erweiterungen und nachfolgende topologische Analysen vorkonfiguriert.

[ ]: